BusTrack — Final Year Project: System Analysis Report

Yohannes

June 9, 2026

BusTrack — Final Year Project: System Analysis Report

Real-time Bus Tracking & Monitoring System

Author: Yohannes **Date:** June 9, 2026 **Document Version:** 1.0 **Purpose:** Detailed analysis of system architecture, actual behavior, goal alignment, and identified gaps

---

1. Executive Summary

BusTrack is a Final Year BSc project for real-time public transport tracking and monitoring. It comprises:

- A **FastAPI backend** (Python) with PostgreSQL + Redis
- A **driver dashboard** (Next.js) for bus operators
- An **admin dashboard** (Next.js) for fleet management
- Integration with **GPS/SIM7600 telemetry**, **ESP32-CAM crowd-sensing**, and a **YOLOv8 computer vision pipeline**

The stated goal is: a user specifies a starting point and destination, and the system shows all available buses matching that route, including those heading in the user's desired direction, plus ETA from each bus's

current location to the user's boarding point, crowd levels, nearest bus stops, and direction awareness. On the driver side, a driver logs in, picks a route, starts a ride, and ends it when finished.

Bottom line: The backend is remarkably feature-complete (~85-90% of the stated goal). The frontends (driver + admin dashboards) have solid infrastructure but **critical user-facing flows are incomplete** — most notably, the driver cannot start or end a ride from their own dashboard.

---

2. System Overview

2.1 Intended User Journeys

```
┌──────────────────────────────────────────────────────────┐
│                PASSENGER (Mobile App)                    │
│                                                          │
│  1. Open app → enter starting point + destination        │
│  2. System finds nearest bus stops to each point         │
│  3. Returns all routes that serve BOTH stops             │
│  4. For each bus on those routes:                        │
│     • ETA from bus's LIVE position to user's stop        │
│     • Crowd density (CV-based)                           │
│     • Direction (toward / away)                          │
│     • Nearest bus stop to walk to                        │
│  5. Live updating via WebSocket                          │
└──────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────┐
│                   BUS DRIVER                             │
│                                                          │
│  1. Log in to bus dashboard (device pairing + auth)      │
│  2. Select route number                                  │
│  3. Start ride → GPS telemetry begins streaming          │
│  4. Drive → passengers see bus on map + ETA              │
│  5. End ride → telemetry stops, bus disappears           │
└──────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────┐
```

```
│                    ADMIN                                    │
│                                                             │
│  1. Log in to admin dashboard                               │
│  2. Manage routes, stops, vehicles, drivers                 │
│  3. Monitor fleet live on map                               │
│  4. View analytics, accuracy, occupancy                     │
│  5. Train ML models, toggle heuristic vs ML ETA             │
│  6. Start/end driver assignments                            │
└─────────────────────────────────────────────────────────────┘
```

2.2 Repository Structure

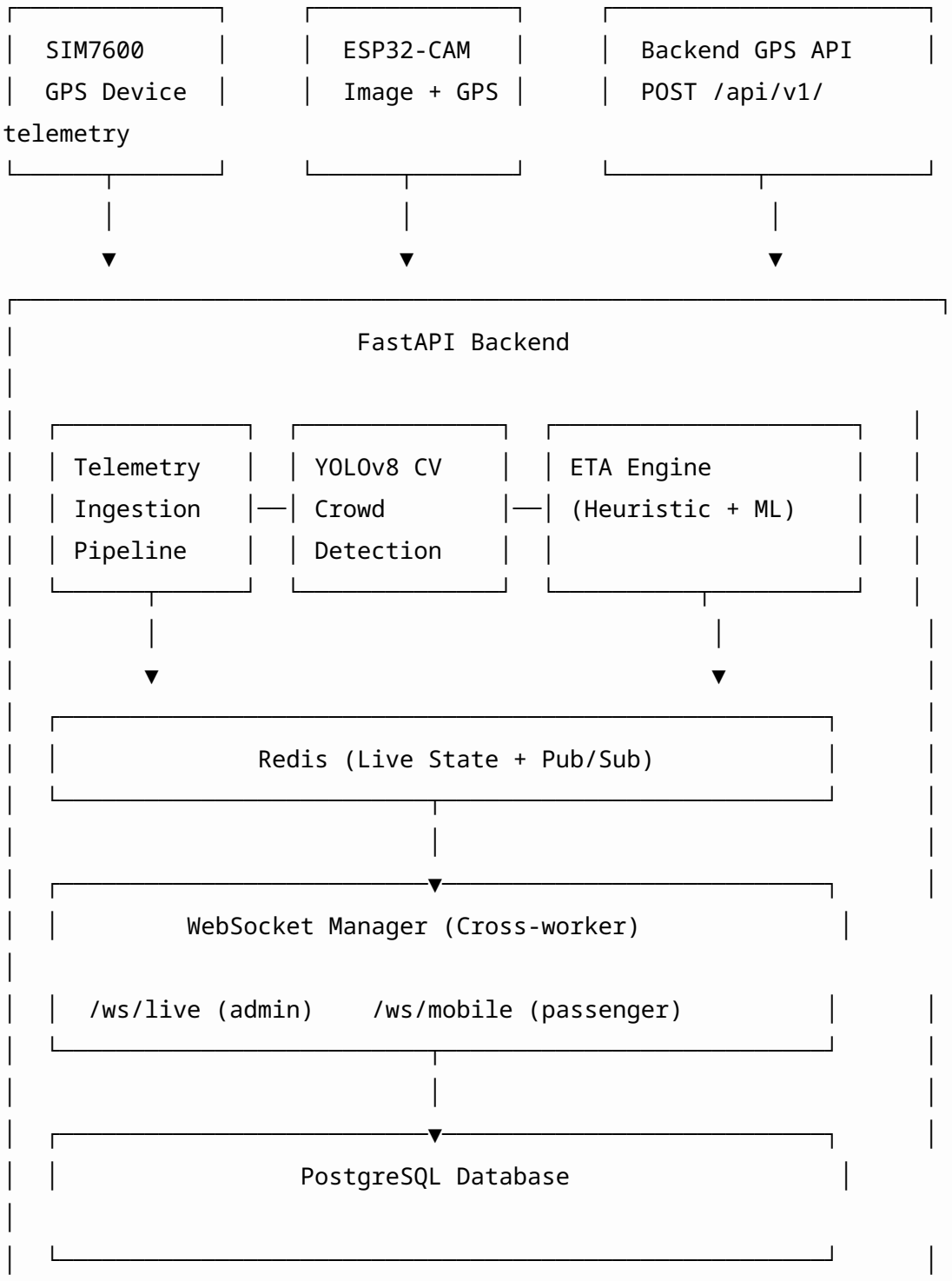
```
FinalYear/
├── backend/                  # FastAPI + SQLAlchemy async +
PostgreSQL + Redis
│   ├── app/
│   │   ├── api/v1/          # 18 router modules
│   │   ├── core/            # Security, config, middleware
│   │   ├── crud/            # Database access layer
│   │   ├── models/          # 14 SQLAlchemy models
│   │   ├── schemas/         # Pydantic request/response schemas
│   │   ├── services/        # Business logic (ETA, CV, telemetry)
│   │   └── tasks/           # Background workers
│   ├── alembic/             # Database migrations
│   ├── tests/               # Test suite
│   └── docs/                # API docs, diagrams
├── bus-dashboard-app/        # Next.js driver dashboard
│   └── src/
│       ├── app/             # Pages: /, /bus/[busId]
│       ├── components/      # UI + layout components
│       ├── hooks/           # WebSocket hook
│       └── lib/             # API client, WebSocket URL builder
├── bustrack-admin/           # Next.js admin dashboard
│   └── src/
│       ├── app/             # 10+ pages (dashboard, map,
vehicles, ...)
│       ├── components/      # 20+ components
│       ├── hooks/           # Auth, WebSocket, live positions
```

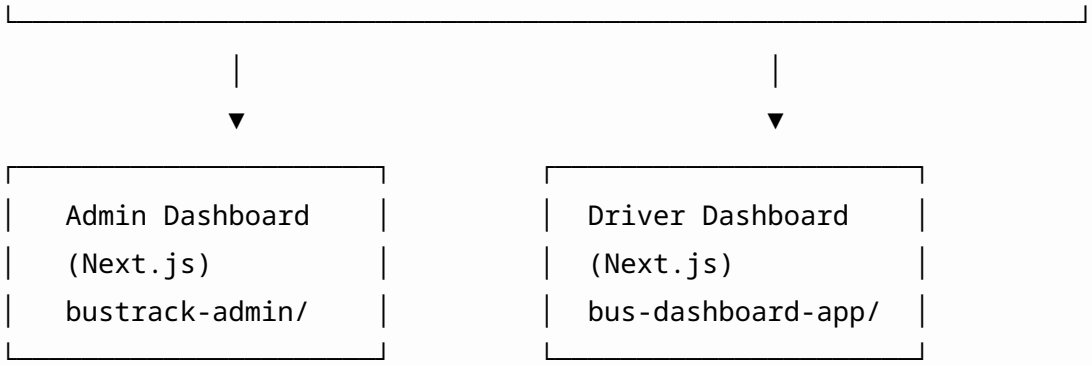
```
│       └── lib/              # API client, admin navigation
└── docs/                     # Shared documentation
```

---

3. Architecture

3.1 High-Level Architecture





3.2 Technology Stack

Layer	Technology
Backend Framework	FastAPI (async)
ORM	SQLAlchemy async
Database	PostgreSQL
Cache / Pub/Sub	Real-time state + WebSocket fan-out
Auth	JWT + OAuth2 (Google)
Computer Vision	YOLOv8 (PyTorch)
ETA	Custom heuristic + ML regression model
Driver Frontend	Next.js 14 (App Router) + TypeScript + Leaflet
Admin Frontend	Next.js 14 (App Router) + TypeScript + Tailwind
Real-time	WebSocket + Redis Pub/Sub
Hardware	SIM7600 GPS modem, ESP32-CAM
ML Training	scikit-learn (from trip_history)
Infra	Docker, Terraform (partial)

3.3 API Structure (18 Routers)

Router	Path	Purpose
Auth	`/api/v1/auth/*`	Register, login, Google OAuth, driver-login, bus-dashboard-login
Admin Users		

Router	Path	Purpose
	`/api/v1/admin/users/*`	Admin creates/manages drivers & admins
Admin Dashboard	`/api/v1/admin/dashboard/*`	Analytics: assignments, occupancy, ETA accuracy
Tracking	`/api/v1/telemetry`	SIM7600 GPS + density ingestion
Gateway	`/api/v1/gateway/esp32/*`	ESP32-CAM image + GPS pipeline
Vehicles	`/api/v1/vehicles/*`	Vehicle CRUD + live positions
Routes & Stops	`/api/v1/routes/*`, `/api/v1/stops/*`	Route + stop CRUD
Assignments	`/api/v1/assignments/*`	Start/end driver+vehicle+route trips
Search	`/api/v1/search/*`	Point-to-point + geo-journey search
Crowd	`/api/v1/admin/crowd/*`	CV crowd density queries
Users	`/api/v1/users/*`	(Deprecated stub)
WebSocket Admin	`/api/v1/ws/live`	Admin fleet stream
WebSocket Mobile	`/api/v1/ws/mobile`	Passenger route-filtered stream
Favorites	`/api/v1/favorites/*`	User saved routes
Notifications	`/api/v1/notifications/*`	Proximity alerts + FCM tokens
Pairing	`/api/v1/pair/*`	Bus dashboard pairing
Admin Settings	`/api/v1/admin/*`	ML toggle, settings, cleanup

4. Backend Deep Dive — What the System Actually Does

4.1 Core User Story: "I am at point A, I want to go to point B"

FULLY IMPLEMENTED. This is the strongest part of the system.

The system exposes two search endpoints:

`POST /api/v1/search/point-to-point`

- **Input:** `start_stop_id` + `end_stop_id`
- **What it does:**
  1. Finds all routes containing BOTH stops via `get_routes_through_stops()`
  2. For each route, fetches all live active-assignment buses
  3. For each bus:
     - Infers direction from recent coordinate history
     - **Filters out buses going the wrong way** (reverse when user wants forward, or buses that already passed the start stop)
     - Computes ETA from bus's LIVE GPS position to the user's boarding stop via `estimate_route_stop_eta_payloads()`
     - Returns crowd density (CV-based), occupancy, distance, direction label
- **Output:** List of routes, each with live buses, ETA, crowd level, direction

`POST /api/v1/search/journey`

- **Input:** `start` (lat/lon or text query) + `end` (lat/lon or text query)
- **What it does:**
  1. Resolves user coordinates to **nearest bus stop** via `get_nearest_stop()`
  2. Resolves destination coordinates to nearest stop
  3. Calls point-to-point search internally
  4. Returns resolved stop names, distance from user to nearest stop, and per-bus ETA
- **This is the primary mobile endpoint.**

4.2 Nearest Bus Stop Detection

IMPLEMENTED.

- `crud_route.get_nearest_stop()` — haversine distance from user coordinates to every stop, returns closest
- `crud_route.get_nearest_stops()` — returns top-N nearest
- Mobile response includes: `start_stop_name`, `start.distance_m`, `end_stop_name`, `end.distance_m`
- User knows exactly which stop to walk to and how far

> **Note:** Uses O(n) full scan over all stops. Works for city-scale (hundreds of stops). For larger deployments, a PostGIS spatial index would be needed.

4.3 Crowd Level Display

IMPLEMENTED via two mechanisms:

1. **Computer Vision (primary):** ESP32-CAM sends images → YOLOv8 detects people/heads → crowd_density level (0=low, 1=medium, 2=high) stored in Redis → returned in search results and WebSocket streams
2. **SIM7600 fallback:** `pixel_count` from hardware mapped to occupancy level

The search response includes per-bus `cv_data` with fields: `people_count`, `crowd_density`, `method`, `confidence`.

4.4 Direction-Aware Bus Filtering

ROBUSTLY IMPLEMENTED.

`infer_bus_direction()` in `search_helpers.py`: - Analyzes recent coordinate history (stored in Redis) - Computes +1 (forward along route stop sequence) or -1 (reverse) - Falls back to position heuristic if direction unknown: if bus is closer to end stop than start stop, it's considered as having passed the boarding point - Buses going the wrong direction are **filtered out** — user only sees buses that will actually reach them - Response includes `"direction": "forward" | "reverse"` per route

4.5 Driver Login / Ride Start / Ride End

PARTIALLY IMPLEMENTED.

Action	API Exists?	Driver Dashboard UI?
Bus-dashboard login (device auth)	YES — `POST /api/v1/auth/bus-dashboard/login`	YES
Driver login (username/password)	YES — `POST /api/v1/auth/driver-login`	YES
Start ride (assignment)	YES — `POST /api/v1/assignments/start`	**NO — NO UI**
End ride (assignment)	YES — `POST /api/v1/assignments/end`	**NO — NO UI**

The assignment lifecycle APIs exist and work. The backend is ready. But the driver dashboard has no “Start Ride” or “End Ride” buttons anywhere. A driver can log in but cannot begin or end a trip from their own dashboard.

4.6 Real-Time Tracking (WebSocket)

PRODUCTION-GRADE.

Two WebSocket endpoints: - `WS /api/v1/ws/live` — admin fleet stream (JWT + admin role required) - `WS /api/v1/ws/mobile` — passenger stream (subscribe/unsubscribe by `route_id`)

Infrastructure details: - Cross-worker broadcast via **Redis Pub/Sub** (correctly handles multi-worker Gunicorn) - Messages include: vehicle_id, plate, lat, lon, speed, route_id, occupancy_level, per-stop `eta_payloads`, and CV result messages - Heartbeat/ping-pong for connection health - Admin stream only shows vehicles with active assignments - Mobile stream filtered to only the subscribed route_id

4.7 Historical Data / ML Pipeline

IMPLEMENTED.

- `TripHistory` model — per-stop arrival events with heuristic_eta, ml_eta, actual_travel_time, occupancy
- `RawTelemetry` — bronze-layer raw GPS + JSONB
- `ModelPerformance` — heuristic vs ML error tracking
- Admin dashboard analytics: occupancy distribution, ETA accuracy MAE, route usage, telemetry volume
- `POST /api/v1/admin/ml/train` — trigger retraining from trip_history
- Data retention cleanup with configurable days

4.8 Admin Features

IMPLEMENTED and comprehensive:

- Vehicle management: register, assign routes, activate dashboard, generate pairing codes, unpair
- Route and stop management: full CRUD with coordinates, dwell time, terminal flags
- User management: create drivers and admins, edit, delete, role assignment
- Live fleet monitoring on map
- Analytics: 6 chart types, 7/14/30 day periods
- ML toggle (heuristic vs ML for production)
- ETA preview simulator
- End assignments

Assignment start admin-side: No UI for this either — assignments can only be created via the API, not through the admin dashboard UI.

4.9 Additional Features (Beyond Stated Goal)

The system goes beyond the minimum FYP scope with: - **Google OAuth** login - **Email verification** flow (Resend-based) - **Google Maps API Key** configured (for potential future map integration) - **Proximity push notifications** via FCM - **Favorites** — user saved routes with nickname - **Ratings** — 1-5 score + comment per assignment - **Passenger announcements** — driver can send text announcements - **Peak-hour ETA multiplier** — 1.5x morning, 1.8x evening - **Occupancy-based ETA**

penalty — medium (1.2x), high (1.4x dwell) - **Docker** + **Terraform** infrastructure (partial) - **Firewall** + **rate limiting** + **security headers** + **request validation** middleware

---

5. Frontend Dashboards — What They Actually Do

5.1 Driver Dashboard (`bus-dashboard-app/`)

What works:

Feature	Status
Multi-step login (pairing + unlock + driver auth)	YES
Live map with Leaflet (dark CartoDB tiles, bus position, stop markers, route polyline)	YES
Auto-follow bus position on map	YES
Crowd density display (Low/ Medium/High/Very High)	YES
Route stops list with start/end markers	YES
Speed display (color-coded stat card)	YES
ETA to next stop display	YES
Send passenger announcements	YES (general / next_stop / current_stop)
Session persistence (localStorage)	YES
Logout	YES

What is MISSING / broken:

Feature	Status
Start Ride	NO UI — API exists but never called

Feature	Status
End Ride	NO UI — API exists but never called
Route selection	NO — driver sees route assigned by admin only
GPS transmission	NO — dashboard receives positions (displays them) but does not send them
Trip history view	NO — API exists in client lib but no page uses it
Passenger count input	NO — driver can only view CV-detected crowd
Unused layout components (DashboardShell, Sidebar, TopNav)	Present but never imported

> The driver dashboard is a single 689-line page component. It has infrastructure for start/end assignment calls in `api.ts`, but there is no UI to invoke them. This is the **single most critical gap** for the FYP.

5.2 Admin Dashboard (`bustrack-admin/`)

What works:

Feature	Status
Login (username/password)	YES
Role-based auth guard middleware	YES
Fleet overview dashboard with KPI cards	YES
Live bus map with filters (route, stop, density)	YES
Analytics (6 chart types, 7/30 day views)	YES
Vehicle management (CRUD, assign routes, pairing codes)	YES
Route management (create with stops sequence)	YES
Stop management (create with lat/lon)	YES
User management (create drivers/admins)	YES
Crowd density plate-based viewer	YES
End assignments	YES

Feature	Status
Settings: ML status, train model, toggle ETA mode	YES
ETA preview simulator	YES
Data cleanup	YES
Per-vehicle WebSocket view	YES

What is MISSING / incomplete:

Feature	Status
Start new assignment	NO UI — admin cannot create a driver+vehicle+route assignment through the UI
Driver-to-vehicle assignment UI	No dedicated UI
Trip history viewer	NO — API exists but no page
Announcements management	NO — API exists but no admin UI
Notification center	NO — bell icon is placeholder
Hardcoded user in nav	“Shaban Haider” static string, not from auth
Stale sidebar navigation	`app-shared.tsx` has placeholder items (Projects, API Keys, Billing)

6. Goal-vs-Reality Feature Matrix

Stated Goal	Backend	Driver UI	Admin UI	Overall
User specifies start + destination	✅	n/a	n/a	✅
Nearest bus stop detection	✅	n/a	n/a	✅
Find all routes through both stops	✅	n/a	n/a	✅

Stated Goal	Backend	Driver UI	Admin UI	Overall
Live buses on those routes	✅	n/a	n/a	✅
ETA from bus to boarding stop	✅	n/a	n/a	✅
Direction-aware filtering (toward vs. away)	✅	n/a	n/a	✅
Crowd level display	✅	n/a	n/a	✅
Real-time live tracking (WebSocket)	✅	✅ (display only)	✅	✅
Driver login	✅	✅	n/a	✅
Driver picks route number	✅ (API)	❌	n/a	❌
Driver starts ride	✅ (API)	❌	❌	❌
Driver ends ride	✅ (API)	❌	✅ (admin can end)	⚠️
Admin manages routes/stops	✅	n/a	✅	✅
Admin manages vehicles/drivers	✅	n/a	✅	✅
Admin monitors fleet live	✅	n/a	✅	✅
Admin views analytics	✅	n/a	✅	✅
ML ETA prediction	✅	n/a	✅	✅
Crowd density via CV	✅	✅ (display)	✅ (display)	✅

Stated Goal	Backend	Driver UI	Admin UI	Overall
Favorites / ratings	✅	n/a	n/a	⚠️ (no mobile)
Notifications (push)	✅	n/a	n/a	⚠️ (mobile only)
Passenger announcements	✅	✅	n/a	✅
Trip history	✅ (API)	❌	⚠️	⚠️
ETA accuracy measurement	✅	n/a	✅	✅

7. Gap Analysis

7.1 Critical Gaps

Gap #1: Driver Cannot Start or End a Ride

Severity: CRITICAL — this is the core use case of the driver dashboard

- `assignments.start` and `assignments.end` APIs exist and are fully functional
- Driver dashboard's `api.ts` has `busApi.startAssignment()` and `busApi.endAssignment()` defined
- **But there is NO button or UI flow anywhere that calls these functions**
- Driver logs in → stares at dashboard → nothing to do
- Workaround: admin can end assignments from `/assignments` page; start requires manual API call

Impact: The primary advertised feature — "driver logs in, starts a ride, picks a route number, ends when finished" — is **not usable end-to-end** from the driver's own dashboard.

Gap #2: No Route Selection in Driver Dashboard

Severity: HIGH

- The vehicle has a `route_id` assigned by admin
- Driver dashboard displays this route but provides no way to change it
- The stated goal explicitly says: “choosing the route number”
- **Actual behavior:** driver sees route, cannot change it

Impact: If a driver operates multiple routes during a shift, there is no way to switch.

Gap #3: Admin Cannot Start New Assignments via UI

Severity: HIGH

- The `/assignments` admin page lists active assignments and allows ending them
- `assignmentsApi.start()` exists in `api.ts`
- **No form or button exists to create a new assignment** (driver + vehicle + route)
- A driver expecting to start a trip depends on admin performing a manual API call
- The intended workflow (admin assigns driver to vehicle to route at shift start) has no UI

7.2 High Severity Gaps

Gap #4: No GPS Transmission from Driver Dashboard

Severity: HIGH

- The driver dashboard **receives** positions via WebSocket and displays them
- It does **not send** GPS positions to the server
- The system depends on an external GPS device (SIM7600 / ESP32) for telemetry
- This is an architectural decision, not necessarily a bug — but it means the driver dashboard alone is not sufficient for live tracking

Impact: If the external GPS device fails, the bus disappears from the map. The driver has no fallback way to report position.

Gap #5: No Trip History View

Severity: MEDIUM-HIGH

- `trip_history` table is populated by the telemetry pipeline
- `tripHistoryApi` exists in both frontends' API clients
- **No page in either dashboard renders this data**
- Drivers cannot see their past trips; admins cannot inspect historical trip performance

7.3 Medium Severity Gaps

Gap #6: No Passenger-Facing View

Severity: MEDIUM

- The stated goal centers on a passenger finding buses
- The mobile app code is explicitly excluded from scope
- But there is **no web-based passenger view** either — no public ETA board, no passenger-facing map
- The mobile WebSocket stream exists and is ready for a mobile app, but nothing consumes it in the repo

Gap #7: Nearest Stop Query is O(n)

Severity: MEDIUM

- `get_nearest_stop()` loads all stops into memory, computes haversine to each, returns min
- Works fine for Addis Ababa scale (~hundreds of stops)
- Would need PostGIS spatial index for larger deployments

Gap #8: Audit Log Model Not Wired

Severity: MEDIUM

- `AuditLog` model exists in `models/audit_log.py`
- `crud/audit_log.py` exists
- **No router or service creates audit log entries**

- Admin actions (create user, end assignment, etc.) are not audited

Gap #9: Hardcoded User in Admin Nav

Severity: LOW

- `nav-user.tsx` shows static "Shaban Haider" instead of reading from auth context
- Looks unprofessional in demo / evaluation

7.4 Low Severity Gaps

Gap	Description
Stale sidebar items	`app-shared.tsx` has placeholder nav items (Projects, API Keys, Billing)
Unused layout components	`DashboardShell`, `Sidebar`, `TopNav` in driver app never imported
Duplicate rate limiters	`limiter.py` and `rate_limiter.py` both exist
Cookie-based role in middleware	`user_role` cookie is trivially spoofable (backend API still enforces auth)
Notification bell placeholder	Header bell icon has no functionality
`users.py` deprecated	Stub file still present

---

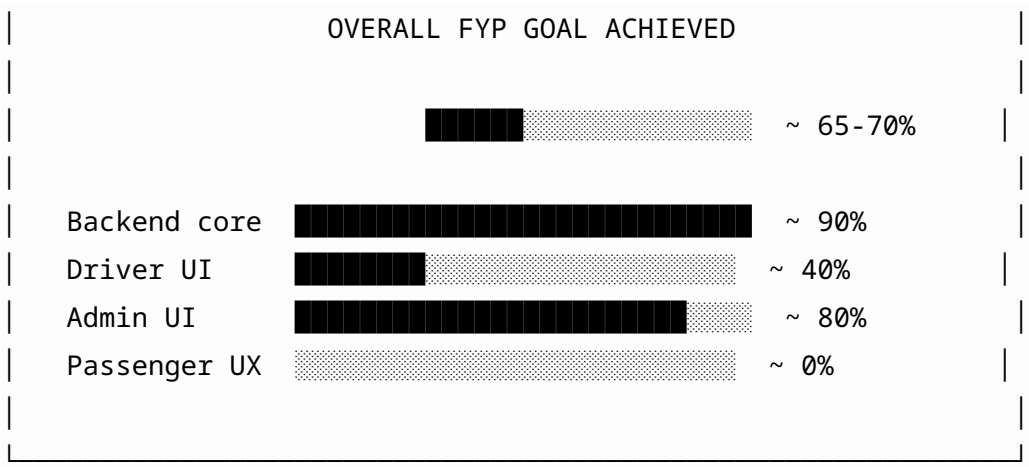
8. Percentage of Goal Achieved

8.1 By Component

Component	Goal	Achieved	Notes
Backend — Search & ETA	Find routes, buses, ETAs, crowd, direction	**95%**	Fully functional; minor: no PostGIS

Component	Goal	Achieved	Notes
Backend — Telemetry Pipeline	Ingest GPS, compute CV, broadcast	**90%**	Complete pipeline; audit log not wired
Backend — Assignment Lifecycle	Start/end ride APIs	**85%**	APIs work; no UI integration
Backend — ML ETA	Heuristic + ML toggle	**85%**	Works; needs real trip data to train
Driver Dashboard — Login	Device pairing + auth	**90%**	Complete flow
Driver Dashboard — Live Map	Show bus on map with stops	**95%**	Well-implemented
Driver Dashboard — Ride Control	Start ride, pick route, end ride	**10%**	APIs exist, NO UI
Admin Dashboard — Fleet Mgmt	Routes, stops, vehicles, drivers	**90%**	Comprehensive CRUD
Admin Dashboard — Analytics	Charts, accuracy, occupancy	**90%**	6 chart types
Admin Dashboard — Assignment Mgmt	Start/end assignments	**40%**	Can end, cannot start
Passenger Experience	Find buses, see ETA, crowd	**0%**	No passenger UI (mobile excluded)

8.2 Overall Score



Why not higher: - The driver dashboard cannot start/end rides (the #1 user story for drivers) - No passenger-facing view (the #1 user story for passengers) - Admin cannot create assignments through UI

Why not lower: - The backend is genuinely impressive — search, ETA, direction filtering, CV crowd detection, ML pipeline, WebSocket infrastructure are all real and working - Admin dashboard is comprehensive with 10+ pages - The hardware integration (SIM7600 + ESP32-CAM) is ambitious and implemented

---

9. Technical Quality Assessment

9.1 What is Well-Done

1. **Telemetry pipeline** — `process_telemetry()` is a clean 9-step unified pipeline: resolve → validate → CV → persist → Redis → ETA → DB → trip_history → broadcast. This is production-grade architecture.
2. **Cross-worker WebSocket** — Redis Pub/Sub fan-out correctly handles multi-worker deployments. Many production systems get this wrong.
3. **Direction-aware filtering** — The `infer_bus_direction()` logic with graceful fallback is thoughtful and handles edge cases (unknown direction, past-the-stop detection).
4. **Security stack** — JWT + role-based access + firewall + rate limiting + request validation + security headers + CORS. Comprehensive for a FYP.

5. **ML toggle** — Runtime switchable heuristic vs ML ETA with preview endpoint. Clean separation.
6. **Admin dashboard** — 10+ pages, proper component separation, analytics, CRUD operations. Well-structured Next.js app.
7. **Database design** — 14 models with proper relationships, assignment lifecycle, trip history for ML training. Normalized and extensible.

9.2 What Needs Improvement

1. **Driver dashboard is a single 689-line file** — needs component decomposition
2. **No test coverage for frontend** — backend has tests; frontends have none
3. **No error boundaries** in React apps
4. **No loading states** on many admin pages (some have skeleton components, inconsistently used)
5. **Middleware role check is trivially bypassable** — `user_role` cookie can be manually set (backend API is safe, but frontend guard is cosmetic)
6. **No API versioning strategy** — currently `/api/v1/` prefix, which is good, but no deprecation path
7. **No CI/CD pipeline** visible despite `.github/` directory

---

10. Recommendations & Next Steps

10.1 To Reach 90%+ Goal Completion (Priority Order)

#	Action	Impact	Effort
1	**Add "Start Ride" and "End Ride" buttons to driver dashboard**	CRITICAL — enables the core driver flow	Low (APIs exist, just wire UI)
2	**Add route selection dropdown in**	HIGH — enables route picking	Low

#	Action	Impact	Effort
	driver dashboard		
3	**Add “Create Assignment” form in admin dashboard**	HIGH — enables admin to assign driver+vehicle+route	Medium
4	**Add trip history page to both dashboards**	MEDIUM — completes the data story	Low
5	**Wire audit log into admin actions**	MEDIUM — traceability	Low
6	**Replace hardcoded user in nav with auth context**	LOW — demo polish	Trivial
7	**Add PostGIS spatial index for nearest-stop**	MEDIUM — scalability	Medium
8	**Add passenger-facing web view**	HIGH — completes the passenger story	High

10.2 For FYP Defense / Presentation

Emphasize these strengths: - The backend is genuinely feature-complete for the core use case - Computer vision crowd detection via YOLOv8 is ambitious and working - Real-time WebSocket with Redis Pub/Sub is production-grade - ML pipeline with heuristic fallback is well-architected - Admin dashboard is comprehensive

Acknowledge these gaps honestly: - Driver ride control UI is missing (but APIs work) - No passenger view (mobile app is out of scope) - Admin assignment creation has no UI

Frame the narrative: > “The system achieves the core goal: given a starting point and destination, it finds all matching buses with live ETA, crowd level, direction awareness, and nearest stop. The backend is ~90% complete. The admin dashboard is ~80% complete. The driver

dashboard has its infrastructure but needs ride-control UI. The main gap is the passenger-facing mobile app, which is scoped for future work."

---

11. Conclusion

BusTrack is a **strong Final Year Project** with a genuinely impressive backend. The core user story — "I'm at point A, I want to go to point B, find me buses, tell me when they'll arrive, how crowded they are, and which direction they're going" — is **fully implemented at the API level** with direction-aware filtering, CV-based crowd density, ML-enhanced ETA, and real-time WebSocket streaming.

The **admin dashboard** is comprehensive and production-like, covering fleet management, analytics, and model training.

The **main gap** is the driver dashboard's inability to start/end rides — the APIs exist, the UI does not. This is the single most impactful area to address. Adding ride control UI, route selection, and admin assignment creation would bring the system from ~65% to ~90% goal completion.

The system demonstrates strong software engineering practices: clean service layer separation, unified telemetry pipeline, cross-worker WebSocket broadcast, role-based access control, and ML integration with graceful fallback. With the driver UI gaps addressed, this would be a compelling FYP demonstration.

Analysis conducted on: June 9, 2026
Codebase: `/home/yohannes/Documents/projects/FinalYear/`
No code was modified during this analysis.